

ハッカソン・フロントエンド開発プレイングガイド

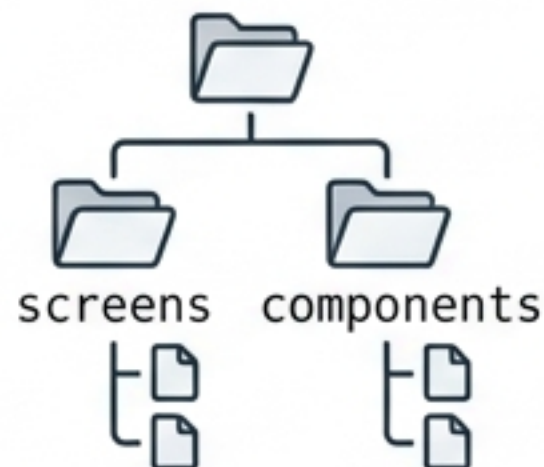
高速かつコンフリクト・ゼロで走り抜けるための開発者用コマンドセンター

```
$ npm run start-hackathon --conflict-free
```


高速開発を支える4つの絶対法則

アーキテクチャ設計

ファイル構造の境界線。
screensとcomponentsの
役割を明確に切り分ける。



不可侵のテリトリー

個人の作業領域。担当画面
ごとのファイル割り当てを厳
守し、作業の衝突を防ぐ。



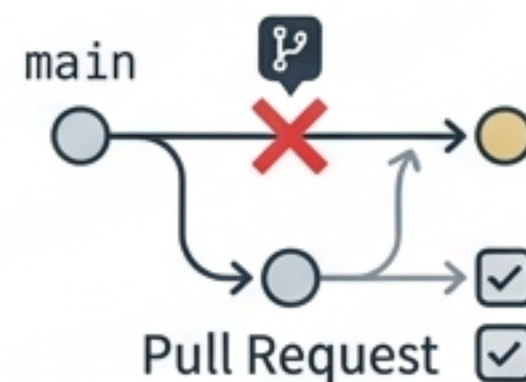
揺るぎない契約

APIコントラクト。
docs/api_contract.mdに基
づくバックエンドとの絶対的
な連携。



安全なGit運用

合流の規律。mainブランチ
への直接コミット禁止と、
正しいPull Requestフロー。



フロントエンドの作業境界線とテリトリー



フロントエンド1

担当: 商品の閲覧と表示

frontend/src/screens/HomeScreen.tsx

frontend/src/screens/ItemDetailScreen.tsx

frontend/src/components/ItemCard.tsx



フロントエンド2

担当: 交換申請のフロー管理

frontend/src/screens/TradeRequestScreen.tsx

frontend/src/screens/RequestListScreen.tsx

frontend/src/screens/RequestDetailScreen.tsx

※バックエンド担当者（API・DB・状態管理）も同様に、Item側（Backend 1）とTrade Request側（Backend 2）で完全にファイルを分割し、1つの共通リポジトリへ合流します。

ディレクトリ構造のメンタルモデル

frontend/

└─ src/

└─ screens/

└─ components/

└─ App.tsx

画面の骨組み

ここにはルーティングに対応する「1画面」を配置する。画面ごとにファイルを分けることでコンフリクトを最小化。

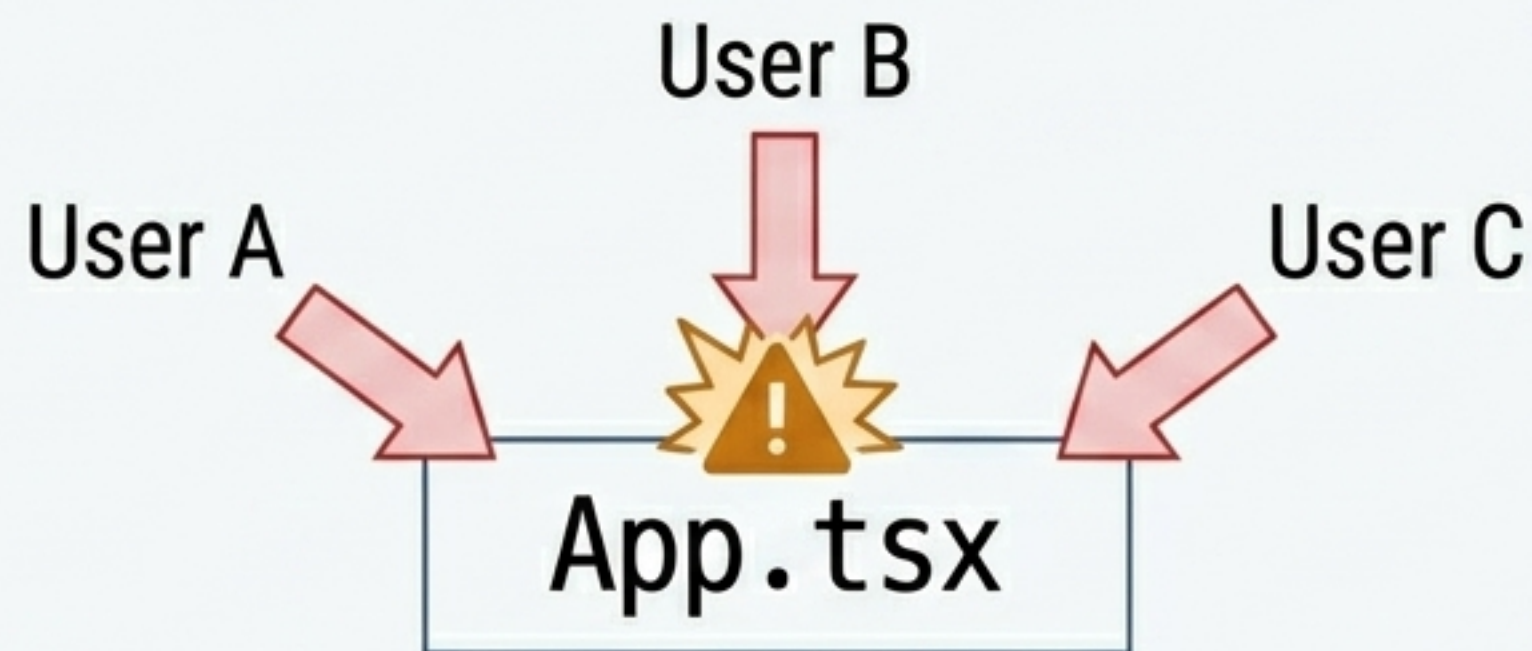
再利用可能なUI部品

ItemCard.tsxなど、複数のScreenから呼び出される独立したパーツを配置。

アプリのルート（要注意）

ルーティング変更などで複数人が同時に触りやすいため、コンフリクト発生リスクが最も高い領域。

最大のコンフリクト発生源「App.tsx」の取り扱い

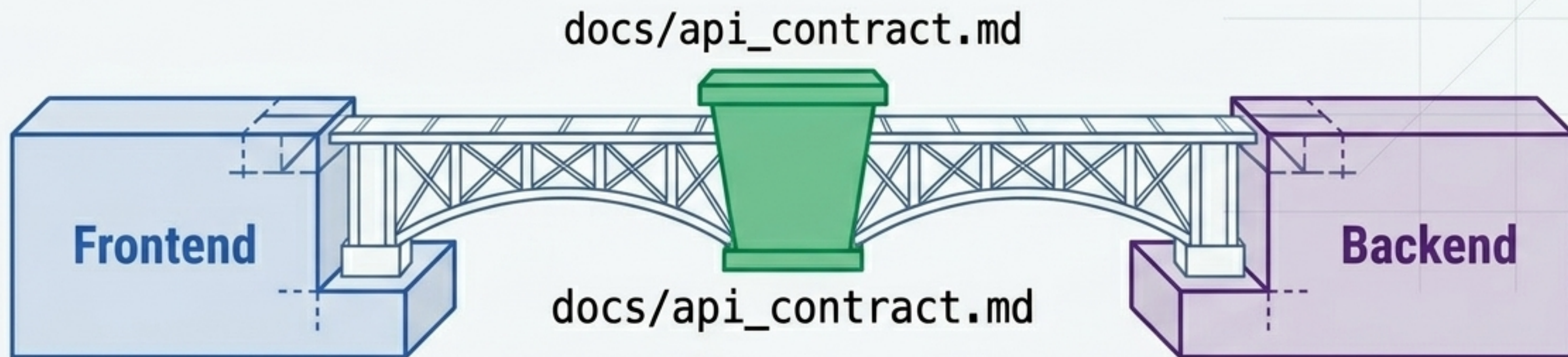


リスク: ルーティング設定やグローバルプロバイダの追加により、全員が同時に編集しがちなファイル。

✓ 回避策（必須ルール）

1. 担当者を一人に決める、または編集する「順番」を明確にする。
2. 画面ごとにファイルを明確に分割し、App.tsx自体のコード肥大化を防ぐ。
3. 同じファイルを複数人で同時に触らない。

APIコントラクト：フロントとバックエンドを結ぶ絶対法



「API仕様はコードを書く前に必ずここに定義する」

役割

フロントとバックの「約束事」をまとめる唯一の情報源。

運用

リーダーは毎日このファイルを確認し、双方の認識ズレを防ぐ。

重要性

バックエンドの実装を待たずに、フロントはこの仕様書を信じてモックデータで開発を進められる。

APIインターフェース設計図：Items（商品管理）

GET

/items

目的: 商品一覧を取得する。

GET

/items/:id

目的: 特定の商品詳細を取得する。

POST

/items

目的: 新しい商品を出品する。

APIインターフェース設計図：Trade Requests（交換申請）

GET

/trade-requests

目的: 交換申請の一覧を取得する。

POST

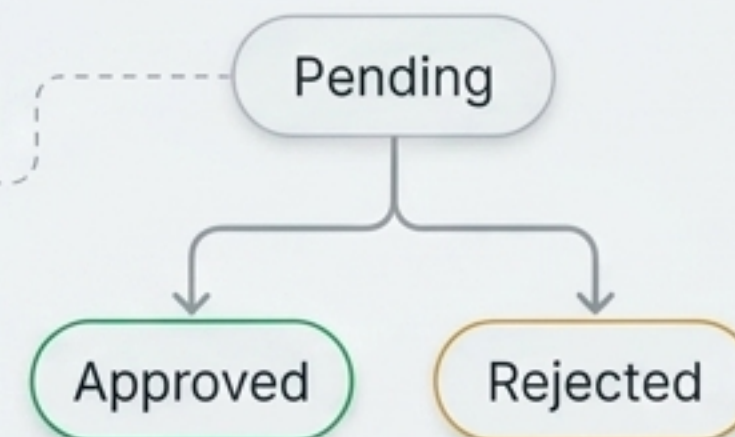
/trade-requests

目的: 新たな交換申請を作成する
(Request Payload必須)。

PATCH

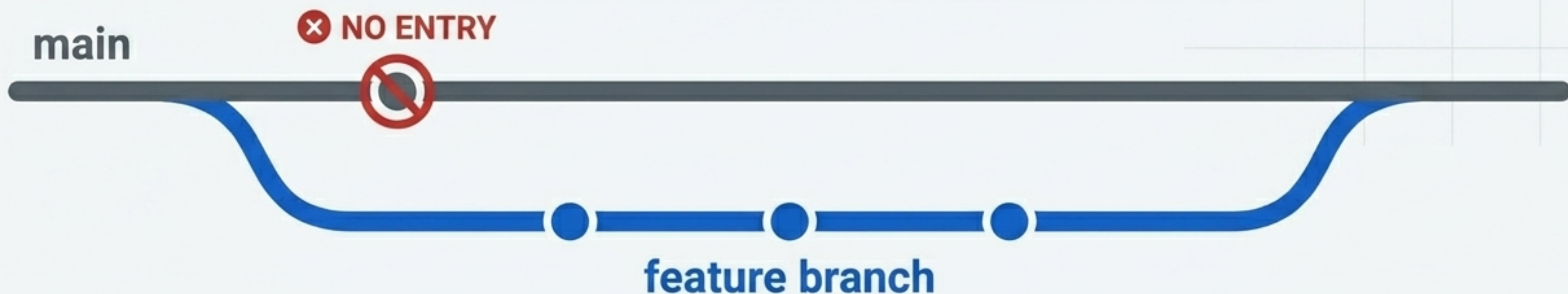
/trade-requests/:id

目的: 交換申請の「状態 (Status)」を
更新する (承認・拒否など)。



Gitブランチ運用における「絶対禁止」事項

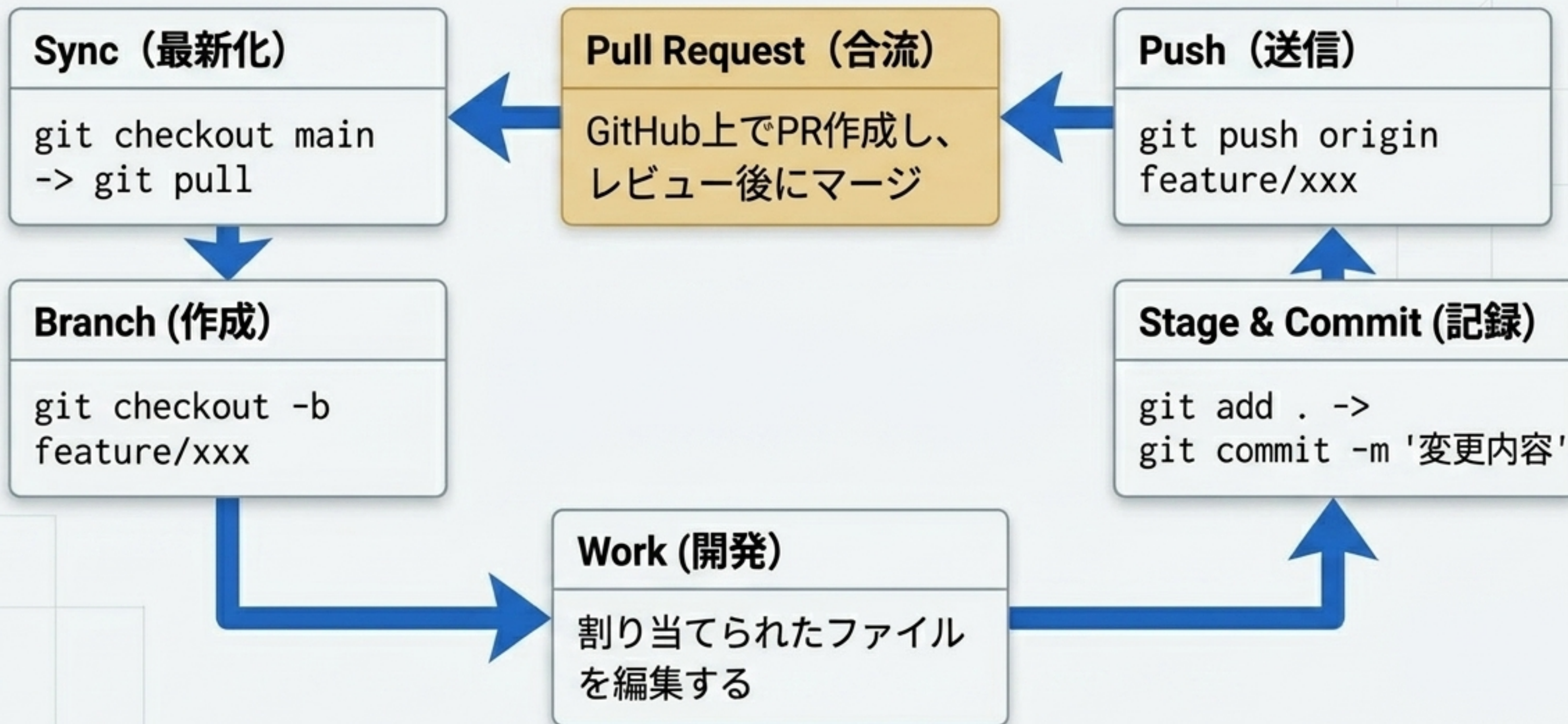
main には絶対に直接作業（コミット）しない。



ブランチ命名規則

- 作業時は必ず専用ブランチを作成する。
- 命名規則例: feature/add-login（機能追加等のわかりやすい名前）

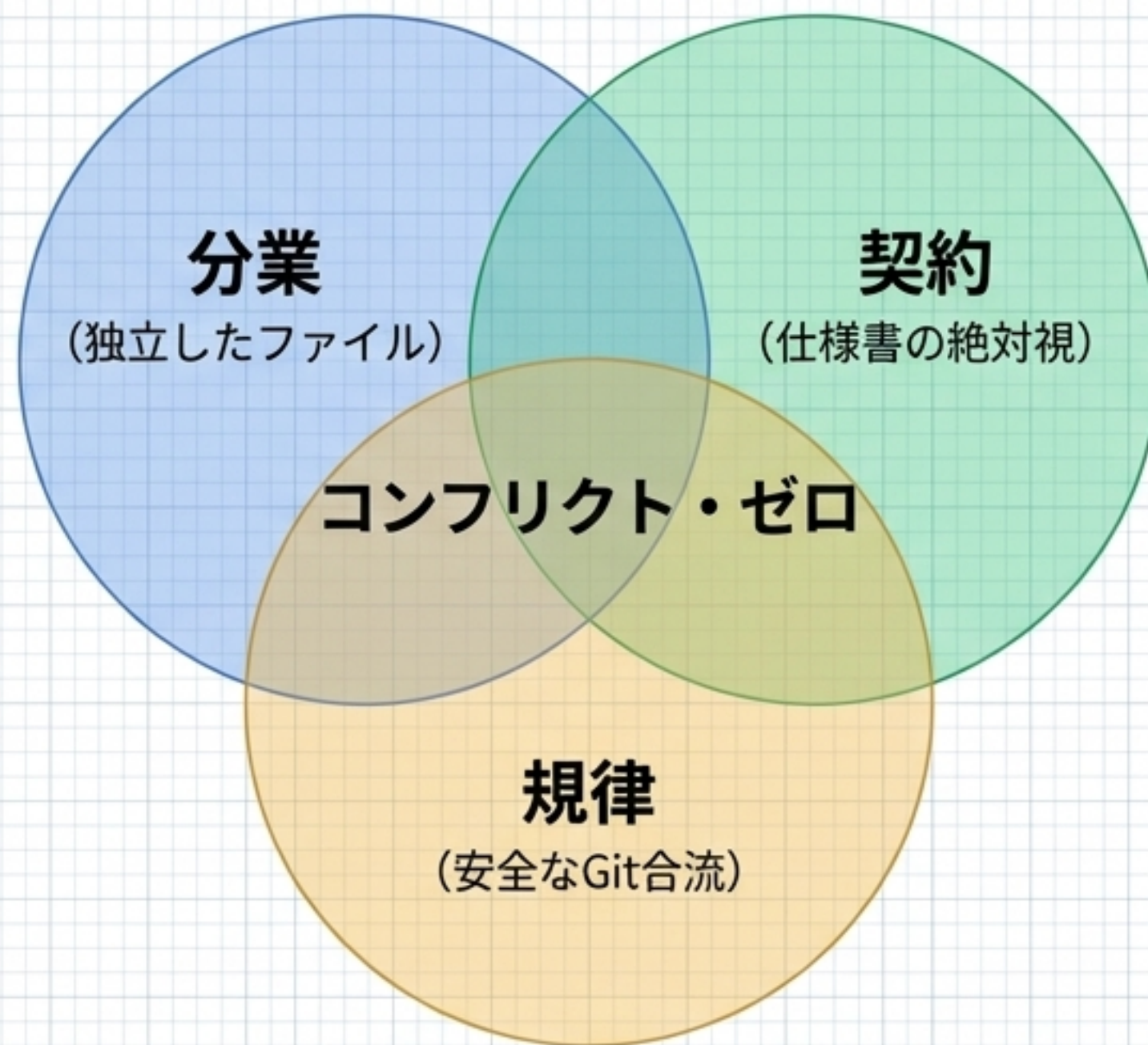
毎日のGitワークフロー・パイプライン



コンフリクト回避・診断マトリックス

Risk (リスク要因)	Mitigation (回避策)
App.tsxでの設定の衝突	作業担当を一人に決めるか、編集タイミングをチーム内で宣言する。
同じコンポーネントの同時編集	画面ごとにファイルを完全に分割し、担当領域を越境しない。
フロントとバックエンドのデータ形式不一致	実装前に必ず docs/api_contract.md に仕様を書き、リーダーが確認する。
古いコードベースでの作業開始	作業開始前に必ず main ブランチに切り替え、git pull を実行する。

高速開発の方程式 (The Zero-Conflict Engine)



ハッカソンでの勝敗は「コーディングの速さ」ではなく、「手戻りと衝突（コンフリクト）をいかにゼロにするか」で決まる。この3つの要素が噛み合うことで、チーム全員が止まることなく並行して最高速で走り続けることができる。

開発者の最終セルフチェックリスト

- ☐ 今から触るファイルは、自分の担当領域 (Territory) か？
- ☐ `App.tsx` を編集する場合、他のメンバーに共有したか？
- ☐ APIの繋ぎ込み時、`docs/api_contract.md` の仕様と完全に一致しているか？
- ☐ 現在のブランチは `main` ではなく、`feature/xxx` になっているか？
- ☐ コミットする前に、不要なファイルを含めていないか確認したか？

```
$ system clear && echo 'Ready for Hackathon.'
```